

K Nearest Neighbors

I. GIỚI THIỆU

Trong lĩnh vực Trí tuệ nhân tạo và Khai phá dữ liệu, các thuật toán phân loại đóng vai trò trung tâm. Trong số đó, k-Nearest Neighbors(k-NN) là một thuật toán học có giám sát thuộc nhóm phi tham số và lười học. Mặc dù có cấu trúc đơn giản, k-NN vẫn chứng minh được hiệu quả vượt trội trong nhiều bài toán thực tế, đặc biệt là các bài toán có đường ranh giới quyết định phức tạp.

Mục tiêu của bài báo cáo này là đi sâu vào bản chất toán học của k-NN thông qua việc cài đặt thủ công thuật toán bằng Python và thư viện NumPy, thay vì sử dụng các "hộp đen" có sẵn như Scikit-learn.

Báo cáo sẽ tập trung giải quyết hai vấn đề chính:

1. Mục tiêu

Hiểu rõ cơ chế hoạt động của thuật toán k-Nearest Neighbors (k-NN).

Tự cài đặt thuật toán k-NN bằng Python (không sử dụng hàm có sẵn của thư viện Scikit-learn).

Áp dụng thuật toán để giải quyết bài toán phân loại trên hai bộ dữ liệu: Iris (Phân loại hoa) và UCI Letter Recognition (Nhận dạng chữ cái).

2. Lý thuyết về k-NN

Định nghĩa: k-NN là thuật toán học có giám sát (supervised learning), thuộc loại (lazy learning) vì không có quá trình huấn luyện mô hình thực sự, mà chỉ lưu trữ dữ liệu.

Nguyên lý hoạt động:

Tính khoảng cách giữa điểm dữ liệu mới cần dự đoán với tất cả các điểm trong tập huấn luyện.

Chọn ra k điểm gần nhất.

Dựa vào nhãn của k điểm này để bầu chọn ra nhãn cho điểm mới.

Công thức khoảng cách: Sử dụng khoảng cách Euclid.

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

II. CÀI ĐẶT THUẬT TOÁN

1. Môi trường thực hiện

Ngôn ngữ: Python 3.13.5 (anaconda)

Thư viện hỗ trợ: NumPy (tính toán ma trận), Pandas (đọc dữ liệu).

2. Mã nguồn

Python

```
def _euclidean_distance(self, x1, x2):
```

```
    # đây là công thức được sử dụng
```

```
    return np.sqrt(np.sum((x1 - x2) ** 2))
```

```
# Hàm dự đoán
```

```
def _predict_one(self, x):
```

```
    # Tính khoảng cách tới tất cả các điểm train
```

```
    distances = [self._euclidean_distance(x, x_train) for x_train in self.X_train]
```

```
    # Lấy k điểm gần nhất và bầu chọn
```

```
    most_common = Counter(k_nearest_labels).most_common(1)
```

Nhận xét: Sử dụng thư viện NumPy giúp việc tính toán khoảng cách giữa các vector (hàng dữ liệu) nhanh hơn so với dùng vòng lặp thường.

III. THỰC NGHIỆM VÀ KẾT QUẢ

1. Bài toán 1: Phân loại hoa Iris

Mô tả dữ liệu:

Số lượng mẫu: 150.

Số đặc trưng (features): 4 (chiều dài/rộng đài hoa và cánh hoa).

Số lớp (classes): 3 loại hoa Iris.

Thiết lập thí nghiệm:

Tỉ lệ Train/Test: 80% - 20%.

Giá trị $k = 3$.

Kết quả:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

=> Độ chính xác nhận diện loài hoa (Accuracy): 100.00%

Độ chính xác (Accuracy): 100 % .

2. Bài toán 2: Nhận dạng ký tự (Letter Recognition)

Mô tả dữ liệu:

Số lượng mẫu: 20,000.

Số đặc trưng: 16.

Số lớp: 26 chữ cái (A-Z).

Xử lý dữ liệu:

Dữ liệu nhãn nằm ở cột đầu tiên.

Đặc trưng nằm từ cột thứ 2 đến hết.

Kết quả:

Thời gian chạy: 97.6163giây.

0	T	2	8	3	5	1	8.1	13	0	6	6.1	10	8.2	0.1	8.3	0.2	8.4
0	I	5	12	3	7	2	10	5	5	4	13	3	9	2	8	4	10
1	D	4	11	6	8	6	10	6	2	6	10	3	7	3	7	3	9
2	N	7	11	6	6	3	5	9	4	6	4	4	10	6	10	2	8
3	G	2	1	3	1	1	8	6	6	6	6	5	9	1	7	5	10
4	S	4	11	5	8	3	8	8	6	9	5	6	6	0	8	9	7

Độ chính xác của nhận diện chữ : 95.30%

Độ chính xác: 95.30 %.

IV. NHẬN XÉT VÀ ĐÁNH GIÁ

1.Về hiệu năng:

Với bộ dữ liệu nhỏ (Iris), thuật toán chạy tức thì.

Với bộ dữ liệu lớn (Letter Recognition - 20,000 dòng), tốc độ dự đoán chậm đi đáng kể. Điều này chứng minh nhược điểm của k-NN là chi phí tính toán lớn ở pha dự đoán (Prediction phase).

2.Về độ chính xác:

Thuật toán hoạt động tốt trên cả hai bộ dữ liệu.

Việc chọn k ảnh hưởng đến kết quả. Nếu k quá nhỏ ($k=1$) dễ bị nhiễu (overfitting), nếu k quá lớn dễ bị mờ nhạt ranh giới giữa các lớp. $k=5$ là một con số hợp lý cho bài toán Letter.

3.Ưu điểm/Nhược điểm rút ra:

Ưu: Dễ cài đặt, không cần huấn luyện .

Nhược: Chậm khi dữ liệu lớn, tốn bộ nhớ lưu trữ toàn bộ tập train.

V. KẾT LUẬN

Đã hoàn thành yêu cầu cài đặt thuật toán k -NN từ đầu và áp dụng thành công cho hai bài toán thực tế. Kết quả thực nghiệm cho thấy độ chính xác cao, phù hợp với lý thuyết đã học.

Kết thúc.